SAÉ S1.01 : Méthodologie d'Implémentation d'un Besoin Client et Conception Algorithmique

Thème principal : Spécification fonctionnelle, décomposition descendante, modularité et programmation par contrat

Synthèse exécutive : Ce cours pose les fondations méthodologiques indispensables du premier projet intégrateur (SAÉ S1.01) du BUT Informatique. Il explique comment transformer un besoin client informel et ambigu en spécifications fonctionnelles rigoureuses, structurer un programme en appliquant la décomposition descendante (Top-Down), séparer strictement la logique métier des interfaces graphiques ou textuelles, et garantir la robustesse du code via la programmation par contrat, les assertions et les tests unitaires.

De l'Expression du Besoin à la Spécification Fonctionnelle

En informatique professionnelle, un projet échoue rarement à cause d'une incapacité technique à coder, mais presque toujours à cause d'un décalage entre ce que le client a exprimé, ce que le développeur a compris, et ce qui a été testé. L'objectif initial d'un projet informatique consiste donc à transformer un énoncé informel en un ensemble de règles explicites et non ambiguës. Toute imprécision lexicale (comme l'utilisation de termes vagues tels que 'rapide', 'ergonomique' ou 'gestion des erreurs') doit être bannie au profit de seuils quantifiables et de comportements déterministes.

- Expression de besoin : formulation subjective et parfois contradictoire fournie par l'utilisateur final
- Spécification fonctionnelle : traduction technique univoque décrivant le comportement attendu du système sans présager de l'implémentation
- Critères d'acceptation : conditions observables permettant d'arbitrer sans équivoque si une fonctionnalité est valide (triptyque 'Étant donné / Quand / Alors')

Cas concret enseignant : *L'enseignant illustre ce problème avec l'analogie classique de la 'balançoire d'entreprise' : le client demande une balançoire suspendue à une branche d'arbre, mais chaque corps de métier (chef de projet, architecte, développeur) l'interprète différemment jusqu'à couper le tronc de l'arbre en deux pour faire passer la corde, alors que le client voulait simplement un pneu attaché avec un câble.*

Décomposition Modulaire et Conception Descendante

Pour concevoir un programme robuste, l'étudiant doit proscrire la rédaction d'un bloc de code monolithique dans une seule fonction 'main'. La démarche requise est la décomposition descendante. Un problème global P est découpé en n sous-tâches P1, P2, ..., Pn. Chaque sous-tâche se matérialise par une fonction dotée d'une responsabilité unique. Il est primordial de séparer systématiquement la logique métier (calculs, algorithmes purs, tris, transformations) de la couche d'interface utilisateur (affichages écran via printf/cout/print et saisies clavier).

- Conception descendante (Top-Down) : démarche consistant à scinder un problème complexe en sous-problèmes élémentaires indépendants
- Modularité : découpage du code en modules autonomes présentant une forte cohésion interne et un faible couplage avec les autres modules
- Signature de fonction : contrat liant l'identifiant de la fonction, la nature de ses arguments d'entrée et son type de retour
- Principe de responsabilité unique (Single Responsibility Principle) : chaque fonction ne doit réaliser qu'une seule tâche précise

Cas concret enseignant : *Métaphore de la chaîne de montage automobile : le technicien qui assemble l'alternateur n'a pas besoin de savoir quelle est la couleur des sièges en cuir. De même, si une fonction de calcul fiscal contient des instructions d'affichage à l'écran, elle devient totalement inutilisable sur un serveur web ou dans une application mobile.*

Programmation par Contrat et Stratégie de Test

Une fonction mathématique ou informatique n'est fiable que si son domaine de validité est formellement délimité. La programmation par contrat stipule que si l'appelant respecte les préconditions, la fonction s'engage à satisfaire les postconditions. Les tests unitaires doivent impérativement couvrir le cas nominal (usage standard), les cas particuliers (données extrêmes, zéros, chaînes vides) et les cas d'erreur attendus.

- Précondition : hypothèse indispensable sur les paramètres d'entrée pour que la fonction garantisse un résultat correct
- Postcondition : état du système et valeur de retour garantis par la fonction une fois son exécution terminée
- Test unitaire : fonction automatisée vérifiant qu'une unité de code (fonction isolée) réagit conformément aux spécifications sur un jeu de données précis
- Assertion (assert) : instruction de contrôle stoppant immédiatement l'exécution en phase de débogage si une précondition est violée

Cas concret enseignant : *Le professeur insiste sur l'accident historique de la fusée Ariane 5 (vol 501 en 1996) : l'explosion a été provoquée par une conversion d'un nombre flottant 64 bits en entier 16 bits sans vérification des préconditions sur la plage de valeurs admissibles, entraînant un dépassement arithmétique et la perte du lancer.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne peut-on pas mettre de 'printf' ou d'affichages dans nos fonctions de calcul d'algorithmes ?
Réponse du professeur : Parce que cela crée un couplage fort avec la console. Si demain vous devez intégrer votre algorithme dans une interface graphique, une application mobile ou un serveur web, votre fonction plantera ou affichera du texte là où il ne faut pas. La logique métier doit être pure et ne faire que des calculs et des retours de valeurs.

Question étudiant : Quelle est la différence exacte entre une précondition et une condition de validation de saisie utilisateur ?
Réponse du professeur : La validation de saisie s'adresse à l'humain (via une boucle 'tant que' qui redemande une valeur si elle est fausse). La précondition est un contrat interne entre développeurs : si l'appelant passe des données aberrantes malgré la saisie (ou via un autre module), le programme doit lever une assertion ou rejeter l'appel.

■ Exercice & Démonstration d'Application

Énoncé : Spécifier et implémenter une fonction modulaire 'calculer_moyenne_ponderee' qui prend en entrée une liste de notes (flottants sur 20) et une liste de coefficients entiers correspondants, puis renvoie la moyenne pondérée.

Démarche de résolution :
Étape 1 (Spécification et Contrats) : - Préconditions : Les deux listes doivent avoir la même taille N > 0. Chaque note doit être comprise entre 0.0 et 20.0. Chaque coefficient doit être un entier strictement supérieur à 0. - Postcondition : Renvoie un flottant compris entre 0.0 et 20.0. Étape 2 (Algorithme) : 1. Initialiser 'somme_points' à 0.0 et 'somme_coeffs' à 0. 2. Parcourir les indices de 0 à N-1 : - Multiplier la note à l'indice i par le coefficient à l'indice i. - Ajouter le produit à 'somme_points'. - Ajouter le coefficient à 'somme_coeffs'. 3. Retourner 'somme_points / somme_coeffs'. Étape 3 (Cas limites) : - Test nominal : notes [10, 14], coeffs [1, 3] -> (10*1 + 14*3)/4 = 13.0.

```
/* ----------------------------------------------------------------
 * Auteur : IUT Anglet - BUT Informatique S1
 * Rôle : Calcul modulaire de moyenne pondérée
 * ----------------------------------------------------------------*/

// CONSTANTES
// (aucune constante nécessaire)

// VARIABLES
// (clavier) >> saisir >> notes, coefficients
// (memoire) >> calculer >> resultat

float calculer_moyenne_ponderee(float tab_notes[], int tab_coeffs[], int taille)
{
    // VARIABLES LOCALES
    float somme_points;
    int somme_coeffs;
    int i;
    float moyenne;

    // TRAITEMENTS
    somme_points = 0.0;
    somme_coeffs = 0;

    for (i = 0; i < taille; i++)
    {
        somme_points = somme_points + (tab_notes[i] * tab_coeffs[i]);
        somme_coeffs = somme_coeffs + tab_coeffs[i];
    }

    moyenne = somme_points / somme_coeffs;

    return moyenne;
}
```

Résultat : La fonction est totalement découplée de toute entrée/sortie console. Elle garantit l'intégrité arithmétique et peut être validée instantanément par une batterie de tests unitaires automatisés.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le risque majeur d'un couplage fort entre le traitement algorithmique et l'affichage console ?

R : L'impossibilité de réutiliser le code dans un autre contexte (interface graphique, application mobile, serveur) et la difficulté à automatiser les tests unitaires.

Q : Qu'appelle-t-on un cas limite (edge case) en conception algorithmique ?

R : Une situation critique aux frontières du domaine de validité (listes vides, tableaux de taille 1, valeurs maximales/minimales) qui provoque souvent des bugs si elle n'est pas traitée spécifiquement.

Q : À quoi sert une assertion (assert) dans un sous-programme ?

R : À vérifier de manière formelle qu'une précondition est respectée pendant l'exécution et à stopper immédiatement le programme en phase de débogage si elle est violée.